BIRDS AI SOUND CLASSIFIER

Prozessüberwachung & Autostart-Architektur (Watchdog-Konzept)

Dokumentation:	autostart_monitor.py & start.bat	**Datum:**	Oktober 2026	**Status:**	**Freigegeben**
Zielanwendung:	birds_ai_sound_classifier.py	**Plattform:**	Microsoft Windows (x64)		

1. Übersicht & Rollenverteilung

Die Skripte "autostart_monitor.py" und "start.bat" bilden ein robustes, aufeinander abgestimmtes Gesamtsystem zur unterbrechungsfreien Ausführung und Überwachung der Bioakustik-Anwendung "birds_ai_sound_classifier.py".

- autostart_monitor.py (Watchdog): Autonome Überwachung aller 60s, Erkennung von Mehrfachstarts, sauberes Beenden von Duplikaten, Prozessbaum-Bereinigung und automatischer Restart.

- start.bat (Starter / Launcher): Sorgt für das korrekte Arbeitsverzeichnis (%~dp0) und öffnet ein separates Konsolenfenster für Statusmeldungen.

- birds_ai_sound_classifier.py: Hauptanwendung mit Flask/Waitress-Webserver, SQLite-Datenbank und ausgelagertem Multiprocessing-Audioprozess.

Komponente	Rolle	Hauptaufgabe
autostart_monitor.py / .exe	Supervisor / Wächter	60s-Loop, Zombie-Bereinigung, Killen von Duplikaten, Neustart
start.bat	Launcher / Wrapper	Setzt Pfad (%~dp0), Start im Konsolenfenster, Pause bei Absturz
birds_ai_sound_classifier.py	Hauptanwendung	Waitress-Webserver, mDNS & Audioanalyse-Multiprocessing

2. Startprozedur & Ausführungskontext

1. **Bevorzugter Aufruf:** In start_app() prüft der Monitor zuerst, ob "start.bat" im Ordner existiert.

2. **Eigenes Konsolenfenster:** Start via "subprocess.CREATE_NEW_CONSOLE". Die Anwendung erhält ein eigenes Fenster für Logs, getrennt vom Monitor.

3. **Arbeitsverzeichnis-Integrität:** "start.bat" wechselt mit "cd /d %~dp0" ins Skriptverzeichnis, damit Modelle, DBs und Configs zuverlässig gefunden werden.

4. **Automatischer Fallback:** Sollte "start.bat" fehlen, startet der Monitor das Python-Skript direkt via "python birds_ai_sound_classifier.py".

3. Erkennung doppelter Instanzen (Duplicate Detection)

Die Erkennung laufender Instanzen erfolgt periodisch über "get_running_instances()" mithilfe der Bibliothek "psutil":

- Filterkriterien: Sucht nach Prozessen mit "python" im Namen und "birds_ai_sound_classifier.py" in der Befehlszeile (cmdline).

- Keine Fehlalarme: Windows startet Multiprocessing-Kindprozesse mit "multiprocessing.spawn". Diese enthalten den Skriptnamen nicht in der Befehlszeile und werden nicht fälschlich als Zweitinstanz gezählt.

- Priorisierung nach Alter (FIFO): Bei mehr als einer Instanz (len(instances) > 1) wird nach "create_time" sortiert: Die älteste Instanz (Index 0) bleibt unberührt, alle neueren werden beendet.

4. Prozess-Terminierung (Main- & Childprozesse)

Beim Beenden überzähliger Instanzen setzt "kill_process_tree()" eine deterministische 5-Stufen-Kaskade um:

Stufe 1 CMD-Fenster merken: Ermittelt das Elternfenster ("proc.parent()"). Ist es "cmd.exe", wird es für das spätere

Schließen registriert.

Stufe 2 Kindprozesse identifizieren: Über "proc.children(recursive=True)" werden alle Subprozesse (Audioanalyse etc.) ermittelt.

Stufe 3 Sanftes Beenden (Graceful): Sendet zuerst an alle Kindprozesse und danach an den Hauptprozess ein "terminate()"-Signal.

Stufe 4 Force-Kill Fallback: Wartet mit "psutil.wait_procs(timeout=3)". Reagiert ein Prozess nicht rechtzeitig, wird er hart via "kill()" abgeschossen.

Stufe 5 Konsolenfenster bereinigen: Schließt abschließend das gemerkte "cmd.exe"-Fenster, um verwaiste Fenster auf dem Desktop zu vermeiden.

5. Schutz vor verwaisten Hintergrundprozessen

Wird die Haupt-App unerwartet beendet ("len(instances) == 0"), führt "cleanup_orphaned_processes()" vor jedem Neustart eine vollständige Bereinigung durch:

- Zombies aufspüren: Sucht im App-Verzeichnis nach verbliebenen Python-Prozessen mit "multiprocessing.spawn" oder Skriptbezug.

- Hardware-Freigabe: Verhindert, dass blockierte PyAudio-Audiostreams (Mikrofon) oder SQLite-Datenbanken den Neustart behindern.

- Eigenschutz: Der Monitor selbst schützt sich durch "autostart_monitor in cmd_str" vor versehentlicher Selbstbeendigung.

6. Betriebs- und Praxishinweise

- Prüfintervall (60s): Die Überwachungsschleife läuft alle 60 Sekunden. Doppelstarts laufen maximal 60 Sekunden, bevor sie eliminiert werden.

- Crash-Verhalten ("pause"): In "start.bat" hält "if %errorlevel% neq 0 pause" das Fenster bei Fehlern offen. Der Monitor startet die App in einem neuen Fenster; das alte Fenster bleibt zur Diagnose sichtbar.

- Autostart-Empfehlung: Ausschließlich "autostart_monitor.exe" (oder .py) in den Windows Autostart ("shell:startup") einbinden. Der Monitor startet und überwacht "start.bat" selbstständig.